

Fuji Manual

v13.5.0-bc956a49f5-mc1.21.11

fuji-fabric

December 26, 2025

Contents

Contents	1
1 Concept	1
1.1 Configuration File	1
1.1.1 Definition	1
1.1.2 Types	1
1.1.3 Example	1
1.2 Module	3
1.2.1 Definition	3
1.2.2 Module Path	3
1.2.3 How to enable/disable a module?	3
1.3 Job	5
1.3.1 Definition	5
1.3.2 Schedule Table	5
1.4 Regex	6
1.4.1 Definition	6
1.4.2 Example	6
1.5 Placeholder	7
1.5.1 Definition	7
1.5.2 Example	7
1.6 Identifier	8
1.6.1 Definition	8
1.6.2 Example	8
2 Command	9
2.1 Definition	9
3 Permission	10
3.1 Definition	10
3.2 Types	10
3.3 What is the permission system used by this mod?	11
3.3.1 Explanation	11
3.3.2 Set a string permission for a command	11
3.4 Reference	11
4 Meta	12
4.1 Definition	12
4.2 Example	12

5	Language	13
5.1	Definition	13
5.2	Modify the default language.	13
5.3	Control how the text is used.	13
5.4	Control the text streaming	14
6	Q&A	15
6.1	Where is the configuration files of fuji?	15
6.2	How can I edit a configuration file?	15
6.3	What is .dat file?	15
6.4	How to update this mod to a new version?	16
6.5	This mod conflicts with one of my mods.	16
6.6	Can I ask the forge or neoforge support?	16
6.7	How can I report bugs or suggest new features?	16
7	Suggestion	17
7.1	Explanation	17
7.2	Useful server-side mods	18

Chapter 1

Concept

1.1 Configuration File

1.1.1 Definition

All files inside `config/fuji/` directory are named **configuration** files.

Tip: List all configuration files.

Issue `/fuji inspect configurations` command.

1.1.2 Types

Note: There are two types of configuration files.

Control File A control file is used to control behaviours.

Main-Control File The **main-control file** is the `config/fuji/config.json` file, which is used to enable/disable a **module**.

Module-Control File Some modules have their own control files, which is used to control the behaviour of that module.

User-Data File A user-data file is used to store the data generated by users.

1.1.3 Example

Example: The main control file.

The main control file is the `config/fuji/config.json` file.
It's applied to **all** modules.

Example: The module control file.

The module control file of "afk" module is the `config/fuji/modules/afk/config.json` file.
It's applied to "afk" module.

1.2 Module

1.2.1 Definition

A module is a self-contained and disable-able unit, to provide a specific function.

Example: The function of modules

The "chat" module, provides chat-related functions.
The "tpa" module, provides /tpa command.

Tip: List all modules.

Issue `/fuji inspect modules` command.

1.2.2 Module Path

A module can be identified by a unique **module path**.

Example: What does a module path look like?

The module path of the module "tpa" is "tpa".
The module path of the module "history" whose parent module is "chat", is "chat.history".

1.2.3 How to enable/disable a module?

To enable an interesting target module, follow the steps:

1. Open the `config/fuji/config.json` file.
2. For the interesting module, set the value of "enable" to be "true" to enable it.
3. Re-start the server to apply the changes.

Warning: Remember to restart the server to apply changes.

You can **NOT** enable or disable a module, while the server **is running**.
After the value of "enable" is modified, you **must** restart the server to apply the changes.
In other words, modules are only loaded during server startup.

Tip: How to enable a sub-module?

To enable the module "chat.display" enabled, you must enable "chat" module first.

Tip: How to know what modules are enabled?

The `/fuji` command is an alias for the `/fuji inspect modules` command.
Issue `/fuji` command, to see what modules you have enabled.

1.3 Job

1.3.1 Definition

A job is some things that will be done **repeatedly**.

Tip: List all jobs.

Issue `/fuji inspect jobs` command.

1.3.2 Schedule Table

The schedule table of a job is a calendar.

A string named **cron expression** is used to **select** time points in the calendar.

When the selected time points are reached, the job will be **triggered/fired**.

When a job is triggered/fired, the things about that job will be done at once.

Example: What does a cron expression look like?

The cron expression "`0 * * ? * *`", means "trigger the job every minute".

The cron expression "`0 */3 * ? * *`", means "trigger the job every 3 minutes".

Tip: Use a generator to make a cron expression.

You can use a generator to select the desired time points: <https://www.freeformatter.com/cron-expression-generator-quartz.html>

1.4 Regex

1.4.1 Definition

A **regex expression** is a **pattern string**, used to match a collection of **input strings**.

1.4.2 Example

Example: What does a regex expression look like?

The pattern string `"cat"` is used to match exactly the given input string `"cat"`.
The pattern string `"cat|dog"` is used to match the given input strings `"cat"` or `"dog"`.

Tip: Test your pattern string on the fly.

Use the following online editor, to test your **pattern string**.

1. <https://regexr.com/>
2. <https://regex101.com/>

1.5 Placeholder

1.5.1 Definition

A placeholder is a string, which will be replaced based on context.

Tip: List all placeholders.

Issue `/fuji inspect placeholders` command.

1.5.2 Example

Example: Use a placeholder to specify the player name.

The placeholder `%player:name%` will be replaced with the name of the contextual player.

Tip: Check the available placeholders

The **PlaceholderAPI** mod is **embedded** in this mod.
By default, it provides a collection of built-in placeholders.
You can add more placeholders by installing additional extension mods.

1. <https://placeholders.pb4.eu/user/default-placeholders/>
2. <https://placeholders.pb4.eu/user/mod-placeholders/>

1.6 Identifier

1.6.1 Definition

An identifier is a string to name an object in Minecraft.

Tip: List all identifiers.

Issue `/fuji inspect registry` command.

1.6.2 Example

Example: The identifier of an item.

The identifier of apple is `"minecraft:apple"`.
The identifier of diamond is `"minecraft:diamond"`.

Example: The identifier of an entity.

The identifier of zombie is `"minecraft:zombie"`.

Chapter 2

Command

2.1 Definition

A `command` is a chat `string` used to perform a specific action/function.

Tip: List all commands.

Issue `/fuji` command, to list all the enabled modules.

Issue `/fuji inspect fuji-commands`, to list all the provided commands by enabled modules.

Chapter 3

Permission

3.1 Definition

A **permission** is used to decide whether a **player** can do something or not.

Tip: List all permissions and metas.

Issue `/fuji inspect permissions-and-metas` command.

3.2 Types

There are two types of permissions.

level permission A level permission is a non-negative integer used in vanilla Minecraft. A higher number means greater authority.

string permission A **string permission** is introduced by a **permission** mod, such as `luckperms` mod.

3.3 What is the permission system used by this mod?

3.3.1 Explanation

Fuji use the Mojang's vanilla Minecraft permission system, which is based on level permission.

As a convention, most of the commands registered by fuji, require level permission to be 0 to use. Only a few of the commands require the level permission to be 4 to use.

Tip: Modify the default required level permission to be 4 for all fuji commands

You may want to prevent the player from using any commands registered by fuji. So that you can assign the proper permission for each command one by one. In this case, you can set the `"all_commands_require_level_4_permission_to_use_by_default"` in `config/fuji/config.json` file to be `true`.

3.3.2 Set a string permission for a command

By default, fuji only use the level permission as the requirement of a command. However, if you want to use string permission for a command, you can use `command_permission` module, to provide luckperms permissions for each command.

3.4 Reference

1. https://minecraft.fandom.com/wiki/Permission_level
2. <https://luckperms.net/wiki/Home>

Chapter 4

Meta

4.1 Definition

A meta is a key-value pair introduced by luckperms mod, to store per-player or per-group variables.

Tip: List all metas.

Issue `/fuji inspect permissions-and-metas` command.

4.2 Example

Example: Set a meta for a group.

The "home" module provides the meta "fuji.home.home_limit", which controls how many homes a player can create.

To set the homes limit to 3: `/lp group default meta set fuji.home.home_limit 3`

Example: List all metas of a group.

Issue `/lp group default info` command.

Chapter 5

Language

5.1 Definition

The files inside **config/fuji/languages/** directory are named **language files**. Each **language file** defines a table to map a **language key** into a **language value**.

Tip: List all languages.

Issue `/fuji inspect languages` command, to see what language files are loaded and in use.

5.2 Modify the default language.

You can modify the **default language** in **config/fuji/config.json** file. Then, issue `/fuji reload` command to apply the changes.

5.3 Control how the text is used.

The **language value** only defines "what is the text". It doesn't define "how the text is used". You can insert **language instructions** to specify how to use that **language value**.

Note: Supported language instructions

1. If the language value starts with "[send-action-bar]", the text will be displayed in the action bar.
2. If the language value starts with "[send-main-title]", the text will be displayed in the main title.
3. If the language value starts with "[send-sub-title]", the text will be displayed in the sub title.
4. If the language value starts with "[suppress-sending]", the text will not be displayed.
5. Otherwise, the text will be displayed in the chat message.

5.4 Control the text streaming

For each fuji command, you can specify "--silent true/false" and "--stdout true/false" flags to control the text streaming during the command execution.

Example:

1. The `/fly others Steve --silent true` command will **not display the text** during the execution of `/fly` command.
2. The `/fly others Steve --stdout true` command will **stream the text into the console** during the execution of `/fly` command.
3. The `/run as console --stdout true send-message @r Hello` command will pass the flags into the `/send-message @r Hello` sub-command.

Chapter 6

Q&A

6.1 Where is the configuration files of fuji?

For hygiene, all the files are placed in `config/fuji/` directory.

6.2 How can I edit a configuration file?

To ensure the readability and compatibility, most of the files are saved as plain text format. You can open them with a text editor. Remember to issue `/fuji reload` command to apply the changes.

Tip: Use a modern text editor.

Some files may have a large number of lines, so it's highly recommended to use a modern text editor, which can highlight symbols and reveal the structure of the file, such as:

1. Visual Studio Code
2. Visual Studio Code - Web Online Editor
3. Vim
4. Emacs
5. Sublime Text

6.3 What is .dat file?

The file whose name ends with `.dat` are the vanilla minecraft NBT format file. To open such a file, you need to use a NBT Editor, such as NBTEditor.

6.4 How to update this mod to a new version?

Please follow the steps:

1. **Backup the data** Back up the `config/fuji` directory.
2. **Test the changes in your test-environment** Put the new version of this mod into `mods/` directory, start the server, test whether everything behaves as expected.
3. **Apply the changes to production-environment** Now, it's ready to apply the changes to your production-environment.

Warning: Never test new changes directly in your production-environment

New changes should always be tested in a test environment, before being deployed to the production environment.

6.5 This mod conflicts with one of my mods.

You can disable the conflicting module and re-start your server. If possible, create an issue at [fuji issue page](#), so that it can be solved later.

6.6 Can I ask the forge or neoforge support?

There are no plans to support other platforms except the fabric platform. However, you can try running this mod via **sinytra-connector** mod. It's tested in the following environment (145/147 modules work, two modules require the `"carpet"` mod doesn't work):

```
Loader: --fml.neoForgeVersion 21.1.216 --fml.fmlVersion 4.0.42 --fml.mcVersion
↔ 1.21.1 --fml.neoFormVersion 20240808.144430 --launchTarget forgeclient
Mods:
- connector-2.0.0-beta.12+1.21.1-full
- forgified-fabric-api-0.115.6+2.1.4+1.21.1
- fuji-fabric-13.2.0-6dbc1fbd06-mc1.21.1
```

6.7 How can I report bugs or suggest new features?

You can create an issue at [fuji issue page](#)

Chapter 7

Suggestion

7.1 Explanation

Here is a collection of server-side mods that are recommended to use, which can make your life easier in fabric server-side admin.

Note that fuji doesn't require these mods installed to work, and some of these mods have the same functionality as fuji.

They are mentioned here so that you know their existence.

7.2 Useful server-side mods

1. [fsit](#)
2. [world threader](#)
3. [blue map](#)
4. [anti xray](#)
5. [melius commands](#)
6. [offline commands](#)
7. [seed guard](#)
8. [spawn anywhere](#)
9. [vanilla permission](#)
10. [vanish](#)
11. [villager config](#)
12. [world game rules](#)
13. [world manager](#)
14. [world-edit](#)
15. [litematica server paster](#)
16. [holo displays](#)
17. [essential commands](#)
18. [missions](#)
19. [luckperms](#)
20. [tab](#)
21. [easy auth](#)
22. [easy whitelist](#)
23. [leukocyte](#)
24. [direction hud](#)
25. [armor stand editor](#)
26. [ban hammer](#)
27. [image2map](#)
28. [polydecorations](#)

29. [sleep warp](#)
30. [styled chat](#)
31. [styled nickname](#)
32. [styled player list](#)
33. [styled sidebar](#)
34. [universal graves](#)
35. [universal shop](#)
36. [patbox's brewery](#)
37. [goml](#)
38. [polydex](#)
39. [polymania](#)
40. [trinkets](#)
41. [head index](#)
42. [inv view](#)
43. [ledger](#)
44. [falling tree](#)
45. [backpacks polymer](#)
46. [double doors](#)
47. [skin restorer](#)
48. [essential addons](#)
49. [husk homes](#)
50. [yet another world protector](#)
51. [krypton](#)
52. [sit](#)
53. [stack deobf](#)
54. [lithium](#)
55. [custom name](#)
56. [poly nutrition](#)
57. [let me despawn](#)

- 58. [carpet](#)
- 59. [mod viewer](#)
- 60. [simple voice chat](#)
- 61. [maintenance mode](#)
- 62. [showcase fabric](#)
- 63. [mini motd](#)
- 64. [tab tps](#)
- 65. [mine spawners](#)
- 66. [spark](#)
- 67. [polyfactory](#)
- 68. [ouch](#)
- 69. [fast back](#)
- 70. [mob filter](#)
- 71. [chunky](#)
- 72. [afk plus](#)
- 73. [taterzens](#)
- 74. [custom name tags](#)
- 75. [pet protect](#)
- 76. [camera boscuro](#)
- 77. [filament](#)
- 78. [sand storm](#)
- 79. [toms mobs](#)
- 80. [secure crops](#)
- 81. [grief defender](#)